PREDICTIVE MAINTENANCE AND REMAINING USEFUL LIFE ESTIMATION OF MOTOR

PROJECT PRELIMINARY REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF DEGREE OF

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

of

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

by

ANAMIKA SEBASTIAN (VAS23EC019)

P. DEVI CHANDANA(VAS23EC053)

SILPA J. MARAR(VAS23EC058)

VIVEK K.V (VAS23EC063)



(AN ISO 9001:2015 CERTIFIED INSTITUTION)

Department of Electronics and Communication

Vidya Academy of Science & Technology

Thalakkottukara, Thrissur - 680 501

(http://www.vidyaacademy.ac.in)

October 2026

Department of Electronics and Communication Engineering
Vidya Academy of Science & Technology

Thalakkottukara, Thrissur - 680 501

(http://www.vidyaacademy.ac.in)

(AN ISO 9001:2015 CERTIFIED INSTITUTION)

Certificate

This is to certify that the preliminary project report titled **"PREDICTIVE MAINTENANCE AND REMAINING USEFUL LIFE ESTIMATION OF MOTOR"** is a bonafide record of the work carried out by **ANAMIKA SEBASTIAN (VAS23EC019), P. DEVI CHANDANA (VAS23EC053), SILPA J. MARAR (VAS23EC058), VIVEK K.V (VAS23EC063) of Vidya Academy of Science & Technology, Thalakkottukara, Thrissur - 680 501 in partial fulfillment of the requirements for the award of Degree of Bachelor of Technology in Electronics and Communication Engineering of APJ Abdul Kalam Technological University, during the academic year 2026-2027. The Preliminary Project report has been approved as it satisfies the academic requirements in the respect of preliminary project work prescribed for the said degree.**

Project Guide & Head of Department

Dr. Premanand B

Prof., Dept. of ECE

Project Co-guide

Dr. Abitha M. A

Asso Prof., Dept. of ECE

Undertaking

We, **ANAMIKA SEBASTIAN (VAS23EC019), P. DEVI CHANDANA (VAS23EC053), SILPA J. MARAR (VAS23EC058), VIVEK K. V (VAS23EC063)** hereby undertake that the preliminary project work entitled **"PREDICTIVE MAINTENANCE AND REMAINING USEFUL LIFE ESTIMATION OF MOTOR"**, is carried out by me independently under the valuable guidance of **Dr. Premanand B**, Prof., and Head of Department and **Dr. Abitha M. A**, Asso Prof., Dept. of Electronics and Communication Engineering, Vidya Academy of Science and Technology, Thalakkottukara, Thrissur, in partial fulfillment of the requirements for the award of degree of **Bachelor of Technology in Electronics and Communication Engineering** of **APJ Abdul Kalam Technological University**, during the academic year 2026-2027.

ANAMIKA SEBASTIAN
P. DEVI CHANDANA
SILPA J. MARAR
VIVEK K. V

Thrissur
October 2026



Institution Vision

To be a premier institution in engineering education and research, nurturing innovative professionals for sustainable societal development.

Institution Mission

M1: Provide quality engineering education that empowers students with strong technical competence and analytical problem solving abilities.

M2: Promote research, innovation, entrepreneurship and industry collaboration to develop sustainable technological solutions.

M3: Develop ethical and socially responsible professionals with leadership qualities, lifelong learning abilities and a commitment to societal well-being.

Department Vision

To be a centre of excellence in electronics and communication engineering, developing innovative and ethical engineers for sustainable technological development.

Department Mission

M1: Impart quality education with a strong knowledge foundation in core concepts, and complex problem-solving skills in electronics and communication engineering.

M2: Facilitate research, innovation, and industry-oriented practices in emerging areas of electronics, communication, and embedded technologies, fostering sustainable engineering solutions.

M3: Develop professional graduates with leadership, ethical values, teamwork, and lifelong learning abilities, with a commitment to societal well-being.

Department PEOs

PEO-1: Graduates will apply strong technical competencies and problem-solving abilities in electronics and communication engineering to develop sustainable solutions to complex real-world challenges.

PEO-2: Graduates will excel in professional practice as innovators, entrepreneurs, educators, and researchers, applying interdisciplinary knowledge and skills in their careers.

PEO-3: Graduates will demonstrate professional ethics, teamwork, and leadership qualities, contributing responsibly to society and sustainable technological development.

Department PSOs

PSO-1: Analyze and design electronic circuits, VLSI systems, and embedded architectures to develop efficient hardware solutions for engineering applications.

PSO-2: Solve real-world electronics and communication engineering problems using appropriate hardware, software, and modern engineering tools while considering performance, safety, and sustainability.

PSO-3: Innovate in emerging areas of electronics and communication engineering while upholding professional ethics and addressing societal needs.

Department POs

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/ develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon,

culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Acknowledgement

During the course of our preliminary project work several persons collaborated directly and indirectly with us. Without their support it would be impossible for us to finish our work. That is why we wish to dedicate this section to recognize their support.

We want to start expressing thanks to our project guide and Head of Department, **Dr. Premanand B**, Prof., Dept. of ECE & project Co-guide, **Dr. Abitha M. A**, Asso Prof., Dept. of ECE, because of their valuable advice and guidance towards this work. We received motivation, encouragement and hold up from them during the course of work.

We would like to express our sincere gratitude to project coordinators **Mr. Anil M**, Asst. Prof., Dept. of ECE and **Ms. Remya K. R**. Asst. Prof., of ECE department for their valuable suggestion and feedbacks.

We are grateful to express our thanks to all the faculty members of our department for their support. We articulate our gratitude to all our friends for their support and help for this work.

We are thankful to our Principal **Dr. Sunitha C**, for their sole co-operation.

Last, but not the least I wish to express my gratitude to God Almighty for His abundant blessings without which this effort would not have been successful.

ANAMIKA SEBASTIAN (VAS23EC019)

P. DEVI CHANDANA (VAS23EC053)

SILPA J. MARAR (VAS23EC058)

VIVEK K. V (VAS23EC063)

October 2026

Seventh Semester (2023 Admissions)

Vidya Academy of Science & Technology

Thrissur – 680 501.

Abstract

Motors keep factories running and households functioning, so when one fails without warning, the consequences add up quickly: machines sit idle, repair bills climb, and work grinds to a halt. Traditional maintenance doesn't handle this well. Waiting until something breaks means reacting too late, while replacing parts on a fixed schedule often throws away components that still have plenty of life left.

This project takes a different approach by building an AI-based predictive maintenance system that spots faults as they develop and estimates how much working life a motor has left, known as its Remaining Useful Life (RUL).The system draws on everyday condition-monitoring signals such as vibration, temperature, and current, along with other operating characteristics. Machine learning models study these readings to pick up on abnormal behavior, identify the type of fault, and forecast how the motor's health is likely to change over time.

The goal is to give an early warning, so maintenance can be planned ahead of time rather than carried out in the middle of a breakdown.By bringing together fault detection, condition monitoring, and RUL estimation, this approach can cut down on unplanned downtime, make motors more reliable, and help schedule maintenance more sensibly. In the long run, it also extends the life and efficiency of the equipment. The system can also be paired with sensors and a microcontroller-based monitoring unit, allowing motor health to be tracked in real time.

Contents

CERTIFICATE

UNDERTAKING

VISION MISSION PEO PSO POs

ACKNOWLEDGEMENT **i**

ABSTRACT **ii**

LIST OF FIGURES **vii**

LIST OF TABLES **viii**

LIST OF SYMBOLS AND ABBREVIATIONS **ix**

1 INTRODUCTION **1**

1.1 Objectives of the Work 1

1.2 Motivation for this work 2

1.3 Methodologies Adopted 3

1.4 Outline of the Report 4

1.5 Summary 5

2 LITERATURE REVIEW **6**

2.1 Paper I: Condition monitoring and fault diagnosis of electrical motors [1] 6

2.2 Paper II: Machinery diagnostics and prognostics with condition-based maintenance [2] 7

2.3 Paper III: Machinery health prognostics from data acquisition to RUL prediction [3] . . . 7
2.4 Paper IV: Data-driven methods for predictive maintenance of industrial equipment [4] . . . 7
2.5 Paper V: LSTM network for remaining useful life estimation [5] . . . 8
2.6 Background studies . . . 8
2.6.1 Signal analysis for motor faults . . . 8
2.6.2 Machine-learning models . . . 8
2.6.3 Datasets . . . 9
2.6.4 Sensing and embedded hardware . . . 9
2.7 Summary of the literature survey . . . 9
2.8 Comparison of maintenance strategies . . . 10
2.9 Research gap . . . 10
2.10 Summary . . . 11

3 METHODOLOGIES FOR THE PROJECT 12
3.1 Block Diagram . . . 12
3.2 Circuit Diagram . . . 14
3.2.1 Power and motor connections . . . 14
3.2.2 MPU6050 to ESP32 . . . 15
3.2.3 DS18B20 to ESP32 . . . 16
3.2.4 ACS712 and the voltage sensor . . . 16
3.2.5 OLED and buzzer . . . 17
3.3 Control Part . . . 18
3.3.1 ESP32 firmware . . . 18
3.3.2 Data format . . . 18
3.3.3 Python application . . . 19
3.4 Power Circuit Part . . . 20
3.4.1 Approximate power budget . . . 20
3.5 Basic Parts and Their Working Principles . . . 21
3.5.1 Choice of ESP32 . . . 21
3.5.2 MPU6050 and vibration sensing . . . 21
3.5.3 DS18B20 temperature sensor . . . 22

3.5.4 ACS712 current sensor 22
3.5.5 Voltage sensor 22
3.5.6 Serial interfaces and communication 22
3.5.7 Data preparation and feature extraction 23
3.5.8 Machine-learning models 23
3.5.9 Dashboard 24
3.5.10 Limitations and safety 25
3.6 Summary 25

4 HARDWARE SETUP OF THE PROJECT 26
4.1 Overview 26
4.2 Introduction 27
4.3 Hardware Setup 27
4.3.1 Power Circuit 27
4.3.2 Control Circuit 28
4.4 Hardware Components 28
4.4.1 DC Motor 28
4.4.2 ESP32 Development Board 28
4.4.3 MPU6050 Vibration Sensor 29
4.4.4 DS18B20 Temperature Sensor 29
4.4.5 ACS712 Current Sensor (20 A) 29
4.4.6 DC Voltage Sensor Module 30
4.4.7 LM2596 Step-Down Converter 30
4.4.8 Other Components 30
4.5 Complete Hardware Setup 30
4.6 Hardware Design Considerations 31
4.6.1 Pin Allocation 31
4.6.2 ADC Limits and Signal Conditioning 32
4.6.3 Grounding, Noise and the Motor 32
4.6.4 Sensor Mounting 33
4.6.5 Safety and Handling 33
4.7 Summary 33

5 Project Status Completed in S7 **34**

5.1 Activities Completed 34

5.1.1 Observations 35

5.1.2 Summary 35

6 Project status to be completed in S8 **36**

6.1 Planned activities 36

6.2 Detailed work plan 36

6.2.1 Completion of the hardware 36

6.2.2 Software development 37

6.2.3 System integration 38

6.2.4 Performance evaluation 38

6.2.5 Improvement and optional extensions 39

6.2.6 Documentation 39

6.3 Risks and mitigation 39

6.4 Future scope 40

6.5 Conclusion 41

BIBLIOGRAPHY **42**

List of Figures

3.1 Block diagram of the proposed predictive maintenance system 13
3.2 Circuit diagram of the proposed monitoring unit 15
3.3 Flow of data through the control part 19
3.4 Power distribution in the monitoring unit 20

4.1 Wiring Diagram 27

List of Tables

2.1 Summary of the literature survey 9
2.2 Qualitative comparison of maintenance strategies 10

3.1 Power and motor connections 15
3.2 Connections between the MPU6050 and the ESP32 16
3.3 Connections between the DS18B20 and the ESP32 16
3.4 Connections of the current and voltage sensors 17
3.5 Fields of the JSON record returned by the ESP32 (example values only) . 18
3.6 Python libraries used in the application 19
3.7 Approximate current consumption of the logic blocks 21

4.1 Signal connections between the sensors and the ESP32 28
4.2 Main specifications of the motor 28
4.3 Main figures of the sensors and the converter (from manufacturer datasheets) 31
4.4 Summary of the pin allocation 32

5.1 Project status completed in the seventh semester 34

6.1 Project status to be completed in the eighth semester 37
6.2 Risks and their mitigation 40

List of Symbols and Abbreviations

ADC	Analog-to-Digital Converter
AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma-Separated Values
DC	Direct Current
FFT	Fast Fourier Transform
GPIO	General Purpose Input/Output
HTTP	Hypertext Transfer Protocol
I^2C	Inter-Integrated Circuit Protocol
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory Network
MAE	Mean Absolute Error
MEMS	Micro-Electro-Mechanical Systems
ML	Machine Learning
MPU	Motion Processing Unit
OLED	Organic Light-Emitting Diode
RMS	Root Mean Square
RMSE	Root Mean Square Error
RUL	Remaining Useful Life
SCL&SDA	Serial Clock/Data Line

Chapter 1

INTRODUCTION

Electric motors quietly do most of the world's mechanical work. Studies of industrial energy use regularly put motor-driven systems at well over half of all electricity consumed in factories. Because they are so common, we tend to forget about them until something goes wrong. A bearing starts to hum, a winding overheats, a shaft drifts out of alignment, and within hours or days a machine that was running perfectly is dead on the floor.

The cost of such a failure is rarely just the price of a new motor. There is lost production, delayed orders, emergency repair labour, spare parts that may not be in stock, and in some situations a real danger to the people working nearby. For this reason, engineers have long tried to find ways of knowing the health of a motor before it fails rather than after.

This project is our attempt to build such a system. It combines low-cost sensing hardware with machine learning so that a motor can tell us, in advance, that it is getting unhealthy and roughly how much working life it has left. This first-phase report covers the hardware half of that idea: how we built the motor test bench, how we measure what the motor is doing, and how we get that information safely into a computer for analysis.

1.1 Objectives of the Work

The overall aim of the project is to design and develop an AI-based predictive maintenance system for motors capable of fault detection and RUL estimation. The specific objectives are:

- To study the common failure modes of electric motors, such as bearing faults, rotor

unbalance, shaft misalignment, winding faults and overheating, and to understand how each shows up in measurable signals.

- To review existing literature on condition monitoring, fault diagnosis and RUL estimation so that the design is grounded in proven approaches.
- To design and build a motor test bench with a power circuit that is safe and protected.
- To select and interface suitable sensors for vibration, temperature, current, voltage and speed.
- To develop a microcontroller-based data acquisition unit that samples the sensors reliably and sends the data to a computer or cloud storage.
- To collect baseline healthy-motor data and, under controlled conditions, data representing developing faults.
- To prepare the groundwork for machine learning models that classify faults and estimate RUL.

In semester S7 the work is limited to the first six objectives with a primary focus on hardware. The machine learning part is planned for S8.

1.2 Motivation for this work

Three observations motivated us to take up this topic. First, the nature of maintenance in many small and medium industries is still reactive. A motor is run until it stops. This is cheap in the short term and expensive in the long term. Even an early warning of a few days can convert an emergency into a planned shutdown.

Second, the alternative of scheduled maintenance is not free of problems either. Replacing bearings every fixed number of months wastes components that still had useful life, and, paradoxically, can introduce faults through improper reassembly. A system that decides based on the actual condition of the machine is both more economical and more reliable.

Third, the tools have become accessible. MEMS accelerometers, hall-effect current sensors and capable microcontrollers now cost a few hundred rupees each, and open

datasets and free machine learning libraries are available to anyone. Ten years ago, building a condition-monitoring system needed expensive proprietary equipment. Today a student team can attempt it. We saw this as an opportunity to learn real skills from the fields of sensors, embedded systems, signal processing and machine learning while working on a problem that has real industrial value.

As students of Electronics and Communication Engineering, the project also lets us work on the part we are strongest at: sensing, signal conditioning, embedded programming and wireless data transfer.

1.3 Methodologies Adopted

The work was carried out in a sequence of stages, each building on the previous one.

Literature survey: We began by reading papers on motor fault diagnosis, vibration analysis and RUL prediction, and by studying public datasets such as MAFAULDA. This helped us decide which parameters to measure and at what sampling rates.

System architecture: A block diagram was drawn showing the motor, sensors, signal conditioning, microcontroller, communication link and the computer used for analysis. The power and control parts were separated deliberately to keep high-current switching away from sensitive measurements.

Component selection: Sensors and the controller were selected on the basis of range, resolution, sampling speed, interface type, availability and cost.

Circuit design and assembly: The power circuit (supply, protection and motor drive) and control circuit (sensors and controller) were designed, assembled on a breadboard first, and then moved to a more permanent arrangement.

Testing and calibration: Each sensor was tested individually and compared against a reference instrument such as a multimeter, a tachometer or a thermometer. The complete system was then run under no-load and loaded conditions to confirm stable data logging.

Data collection: Healthy-motor data was recorded to build a baseline. This forms the starting point for the machine learning stage in the next semester.

1.4 Outline of the Report

This report is organized into six chapters, detailing the design, implementation, and future roadmap of the predictive maintenance system:

- **Chapter 1: Introduction** outlines the problem of motor failures in industrial settings, sets the overall project goals, explains the motivation behind choosing low-cost condition monitoring, and provides the methodology adopted during the seventh semester.
- **Chapter 2: Literature Review** examines existing research on motor fault diagnosis, signal processing techniques, and machine learning models for Remaining Useful Life (RUL) estimation, highlighting the research gap that this project aims to address.
- **Chapter 3: Methodologies for the Project** presents the system architecture, detailing the block diagram, circuit schematic, sensor selection, signal conditioning strategy, data flow through the ESP32 firmware, and the Python-based machine learning pipeline.
- **Chapter 4: Hardware Setup of the Project** describes the physical construction of the test bench, including the motor setup, power distribution via the step-down converter, pin assignments, sensor mounting, and noise reduction techniques.
- **Chapter 5: Project Status Completed in S7** summarizes the work completed during the seventh semester, focusing on the successful hardware assembly, sensor verification, and wireless data streaming over the local Wi-Fi network.
- **Chapter 6: Project Status to be Completed in S8** presents the planned schedule for the eighth semester, detailing the current channel calibration, fault dataset generation, Random Forest model training, live RUL estimation, and dashboard integration.

1.5 Summary

This chapter introduced predictive maintenance as a response to the cost and unpredictability of motor failures. It listed the objectives of the project, explained why we chose it, and described the approach followed in S7. The next chapter reviews the existing work on which this project builds.

Chapter 2

LITERATURE REVIEW

Our starting question was simple: how do people catch a motor fault before the motor stops, and how do they judge how much life is left in it? To answer it we read a fair number of papers, and this chapter discusses the five that shaped our work most. A few background references on signal analysis, machine learning and datasets come after them. At the end we compare the usual maintenance strategies and say where we think the gap is.

2.1 Paper I: Condition monitoring and fault diagnosis of electrical motors [1]

The review by Nandi *et al.* [1] was our first stop, because it lists what actually goes wrong in a motor: bearing damage, broken rotor bars, stator winding faults and eccentricity. For each of them it explains which signal shows the trouble first. Vibration is good for bearings, current for winding and rotor problems, and temperature tells you about heating in general.

What we took from it is that one sensor is never enough. A fault that is invisible in one signal can be clear in another, and this is why our rig measures vibration, current, voltage and temperature together. The paper deals only with finding faults, though. It says very little about how long the motor can keep running afterwards, and that question is the one we care about. It supports the first objective in Section 1.1.

2.2 Paper II: Machinery diagnostics and prognostics with condition-based maintenance [2]

Jardine *et al.* [2] look at the whole condition-based maintenance process, from collecting data to deciding when to act. They also make a distinction that we found useful. Diagnosis is about a fault that has already happened. Prognosis is about what will happen next, either predicting a fault early or estimating the time left before failure.

By their account, diagnosis is in good shape but prognosis is still difficult, partly because it needs data from machines that were run until they failed, and such data is hard to get. We follow their order of work in our own pipeline: acquire, extract features, detect and classify, then estimate remaining life.

2.3 Paper III: Machinery health prognostics from data acquisition to RUL prediction [3]

This is a long review by Lei *et al.* [3], and it is the one we went back to most often when planning the RUL part. They break prognostics into four steps: data acquisition, building a health indicator, dividing the health into stages, and predicting the RUL. Physics-based, statistical and AI-based models are all discussed, with the point that the quality of the health indicator decides most of the final accuracy.

They also warn that many methods look good on a laboratory dataset and then struggle on real machines. We kept that warning in mind when we decided to test on our own rig as well as on public data.

2.4 Paper IV: Data-driven methods for predictive maintenance of industrial equipment [4]

Zhang *et al.* [4] compare machine-learning and deep-learning methods used for fault detection, classification and RUL. Their conclusion is fairly practical. Data-driven models work well, but only when the data is clean and the features are chosen with care, and very few of the methods they surveyed were ever run in a real-time monitoring set-up.

That last point is what pushed us toward a small microcontroller-based unit instead of working only on saved datasets.

2.5 Paper V: LSTM network for remaining useful life estimation [5]

Zheng *et al.* [5] feed multi-sensor time series into an LSTM network to estimate RUL. Because the network remembers earlier readings, it can follow the slow drift of a degrading machine, which a model that sees one reading at a time cannot do. On a standard turbofan engine dataset, they report better results than the conventional methods they compared against.

The catch is cost. Training such a network needs a lot of run-to-failure data and a good deal of computing, and we have neither. So we treat this paper as a reference for where RUL modelling can go, and we begin with a health indicator and simpler regressors, with an LSTM left as an option.

2.6 Background studies

Some other works did not need a full section but still shaped what we built.

2.6.1 Signal analysis for motor faults

The FFT is the standard tool for finding fault frequencies in vibration and current [1]. Park's Vector turns the three-phase currents into a two-dimensional curve. For a healthy motor this is close to a circle, and a fault bends it out of shape, as Cruz and Cardoso [6] showed for stator winding faults. Both ideas give us features for the classifier.

2.6.2 Machine-learning models

Random Forest [7] is a good fit for a small tabular dataset because it needs little tuning. For degradation over time, LSTM networks [8] are the usual choice. Lei *et al.* [9] also remark that in fault diagnosis the features often matter more than the model, which matched our own experience.

2.6.3 Datasets

MAFAULDA [10], from the Federal University of Rio de Janeiro, has multi-channel signals for several faults and severities on a machinery fault simulator. For run-to-failure work, PRONOSTIA [11] and C-MAPSS [12] are common. We use MAFAULDA for fault classification and plan to check the method on our own motor afterwards.

2.6.4 Sensing and embedded hardware

The rig is built around an ESP32-S3 with an ADXL345 accelerometer [13] and sensors for current, voltage and temperature. The datasheets set the sampling limits and interfaces, which we use in Chapter 3.

2.7 Summary of the literature survey

Table 2.1 lists the five papers with their methods and what each leaves unsolved.

Sl.	Paper details	Methodology / key findings	Research gap / limitations
1	Nandi *et al.* (2005) [1]	Review of motor faults and the signals and techniques used to detect them	No single signal covers all faults; focus on detection, not remaining life
2	Jardine *et al.* (2006) [2]	Framework of diagnostics and prognostics for condition-based maintenance	Prognosis is less mature than diagnosis
3	Lei *et al.* (2018) [3]	Health indicator construction, stage division and RUL prediction	Mostly laboratory data; hard to carry to real equipment
4	Zhang *et al.* (2019) [4]	Survey of machine-learning and deep-learning maintenance methods	Needs good data; few real-time implementations
5	Zheng *et al.* (2017) [5]	LSTM for RUL from multi-sensor time series	Needs large run-to-failure data and heavy training

Table 2.1: Summary of the literature survey

2.8 Comparison of maintenance strategies

Table 2.2 compares the common strategies on points a workshop or factory would care about. It is a qualitative picture drawn from the surveys [2, 4] and is not based on measured data.

Criterion	**Reactive**	**Preventive (scheduled)**	**Condition-based**	**Proposed system**
When action is taken	After failure	At fixed intervals	When a fault appears	Before failure, based on RUL
Unplanned downtime	High	Reduced	Reduced	Reduced (target)
Parts with life left discarded	No	Often	Rarely	Rarely
Fault type identified	No	No	Sometimes	Yes
Remaining life estimated	No	No	Rarely	Yes
Data needed	None	Little	Sensor data	Sensor data and models

Table 2.2: Qualitative comparison of maintenance strategies

2.9 Research gap

Reading these papers one after another, three gaps kept showing up.

1. Most work stops at detecting or classifying a fault. Very few studies put detection, classification and RUL estimation into one system.

2. Many methods are tested only on public or laboratory datasets. Hardly any are tried with a cheap sensing unit that measures vibration, current, voltage and temperature on a real motor.

3. The deep models used for RUL ask for more data and computing than a small team with low-cost hardware can provide.

Our answer is a system that joins fault detection, fault classification and RUL estimation, built on a low-cost microcontroller unit with a simple dashboard. We kept the design modest on purpose, so that it can be built and repaired with parts that are easy to find.

2.10 Summary

We went through five main papers and a handful of background references on signal analysis, learning models and datasets. Fault diagnosis is well covered, prognosis is still developing, and few works link a low-cost sensing unit to a full detection, classification and RUL pipeline. The next chapter describes the method we use to fill that gap.

Chapter 3

METHODOLOGIES FOR THE PROJECT

This chapter explains how the proposed system works. We first show the overall architecture as a block diagram and the electrical connections as pin tables. Then we describe the control part, which is the firmware on the ESP32 and the Python application on the computer, and the power part. The last section explains the basic ideas behind the main building blocks: the sensors, the ESP32 interfaces, the feature extraction, the two machine-learning models and the dashboard.

3.1 Block Diagram

Figure 3.1 gives the block diagram of the system. The motor under test is a 12 V brushed DC motor. Four sensors are attached to it, and all of them feed one ESP32 board. The ESP32 shows the status locally on an OLED display, sounds a buzzer when something looks wrong, and passes the readings on to a dashboard. The Python application and the trained models run on a computer, which we show as a dashed block because the ESP32 itself does not run the models in the first version.

Here is what each block does.

12 V DC motor: This is the machine we want to watch. It is a Yalu MY6812 brushed DC motor, rated at about 150 W and 2750 RPM. We take these figures from earlier project notes, and they will be checked against the nameplate before the final report.

MPU6050: A three-axis accelerometer and three-axis gyroscope on one chip. It is fixed

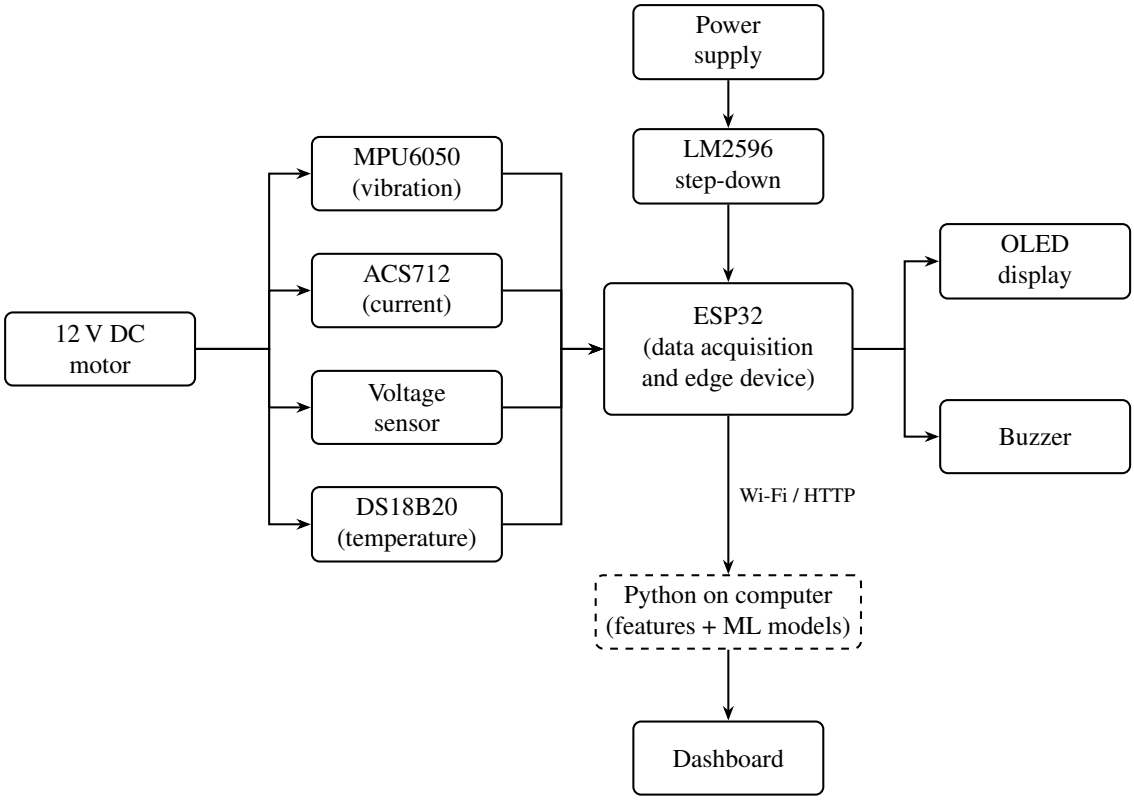


Figure 3.1: Block diagram of the proposed predictive maintenance system

to the motor body and gives the vibration signal, which is our main source of information about mechanical faults.

ACS712 (20 A): A Hall-effect current sensor placed in series with the motor supply. Changes in load and abnormal current show up here.

Voltage sensor: It measures the motor supply voltage, so that we can tell a change in motor behaviour from a change in the supply.

DS18B20: A digital temperature sensor mounted on the motor housing, used to follow heating during operation.

ESP32: It reads all four sensors, puts the readings into one record and offers them over Wi-Fi. It also drives the OLED and the buzzer. In this first version it does not run the machine-learning models, because the Python side is easier to train, change and debug.

Power supply and LM2596: A DC supply feeds the motor, and the LM2596 step-down converter brings the voltage down to 5 V for the ESP32 board.

OLED display and buzzer: The OLED shows live values and the status, and the buzzer gives a local warning, so the unit is useful even when nobody is looking at the dashboard.

Python on the computer: It collects the data, saves it, extracts features, runs the trained models and returns the results.

Dashboard: A web page that shows the live readings and, once the models are trained and checked, the fault class and the remaining useful life.

The ESP32 and the computer talk over a local Wi-Fi network created by the ESP32 itself, so no internet connection is needed. The same architecture can later be reduced to a fully stand-alone unit by moving a small model onto the ESP32, and we keep this as a possible extension.

3.2 Circuit Diagram

Figure 3.2 shows the complete wiring of the monitoring unit. A single 12 V supply feeds both the motor and the electronics. The motor current passes through the ACS712 and a switch, and an LM2596 step-down module makes the 5 V rail for the ESP32 and the sensors. In the figure, red wires are positive connections and black wires are ground. The coloured wires are signals: green for the ACS712 output, blue for the voltage-sensor output, purple for the I2C data line (SDA), dark blue for the I2C clock (SCL) and cyan for the DS18B20 data line. These are the proposed connections. Before we power the circuit, each module is checked against its own datasheet, because breakout boards from different makers do not always follow the same pinout or voltage levels. The tables below list the connections one by one, and the complete pin map is in Table 4.1.

3.2.1 Power and motor connections

The positive terminal of the 12 V supply goes to three places: one terminal of the ACS712, the IN+ terminal of the LM2596 and the input terminal of the voltage sensor. The other terminal of the ACS712 goes through the switch to the positive lead of the motor. The ACS712 therefore sits in the motor path and carries the full motor current. Because the LM2596 is connected before the switch, the motor can be stopped without switching off

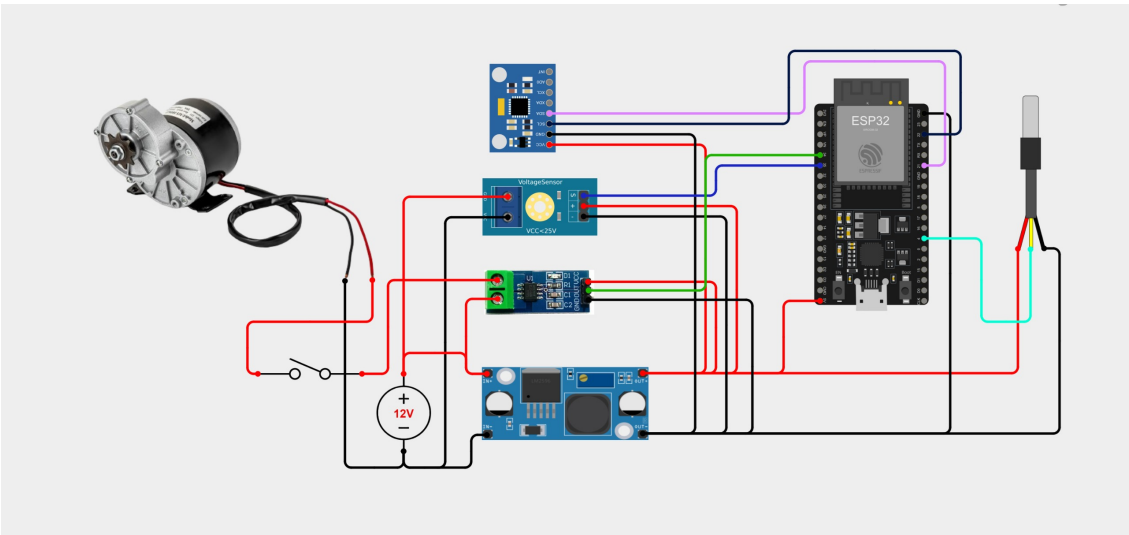


Figure 3.2: Circuit diagram of the proposed monitoring unit

the ESP32. The negative terminal of the supply goes to the negative lead of the motor, the IN− terminal of the LM2596 and the voltage sensor.

The output of the LM2596 is set to 5 V with its trimmer and checked with a multimeter *before* the ESP32 is connected. This 5 V output feeds the 5V pin of the ESP32 and the sensor modules, and the output ground of the LM2596 is the common ground of all of them. On the usual LM2596 module the IN− and OUT− terminals are joined on the board, so the motor side and the logic side share one ground. We confirm this with a multimeter.

Point	**Connected to**
12 V supply (+)	ACS712 terminal, LM2596 IN+, voltage sensor input (+)
12 V supply (−)	Motor negative lead, LM2596 IN−, voltage sensor input (−)
Other ACS712 terminal	Switch, then motor positive lead
LM2596 OUT+ (5 V)	ESP32 5V pin, MPU6050 VCC, ACS712 VCC, voltage sensor +, DS18B20 VDD
LM2596 OUT−	ESP32 GND and the GND of all sensors

Table 3.1: Power and motor connections

3.2.2 MPU6050 to ESP32

The MPU6050 talks over I2C. In the diagram it is powered from the 5 V rail, which is allowed because the breakout board has its own 3.3 V regulator. The voltage on the SDA and SCL lines is checked once to confirm that it stays at 3.3 V. The AD0 pin is left

unconnected, and the board pulls it low, so the module answers at address 0x68, which is also the address it was seen at during our earlier tests. The INT, XDA and XCL pins are not used.

MPU6050 pin	ESP32 / circuit	Purpose
VCC	5 V rail (LM2596 OUT+)	Power (on-board regulator)
GND	GND	Common ground
SDA	GPIO 21	I2C data
SCL	GPIO 22	I2C clock
AD0	Not connected	Address 0x68 (pulled low on the board)

Table 3.2: Connections between the MPU6050 and the ESP32

3.2.3 DS18B20 to ESP32

The DS18B20 uses a one-wire data line. The probe has three wires: red is VDD, black is GND and yellow is DATA. VDD is taken from the 5 V rail and DATA goes to GPIO 4. A 4.7 kΩ pull-up resistor from DATA to 3V3 is needed. It is not drawn in the figure and will be added on the board. We connect it to 3V3 and not to 5 V, so that GPIO 4 never sees more than 3.3 V. The wire colours of the probe are checked because they are not the same for every maker.

DS18B20 wire	ESP32 / circuit	Purpose
VDD (red)	5 V rail (LM2596 OUT+)	Power
GND (black)	GND	Ground
DATA (yellow)	GPIO 4	One-wire data, 4.7 kΩ pull-up to 3V3

Table 3.3: Connections between the DS18B20 and the ESP32

3.2.4 ACS712 and the voltage sensor

The ACS712 is the only part that sits in the motor's power path. The motor current flows through its two screw terminals, so the sensor is wired in series with the motor: one terminal to the positive of the supply and the other through the switch to the motor. Its OUT pin carries only a small measurement voltage to the ESP32 and none of the motor current. The voltage sensor measures the supply voltage, so its input terminals are connected across the 12 V supply.

The ESP32 ADC inputs are not 5 V tolerant. Both modules run from the 5 V rail, and their outputs can go above what the ESP32 accepts. The output of the ACS712 rests at about 2.5 V at zero current and changes by about 0.1 V per ampere, so it would pass the 3.3 V limit at roughly 8 A. This is below the rated current of our motor, so a voltage divider is likely to be needed, and the offset and sensitivity used in the calibration must then include the divider. The output of each module is measured with a multimeter first, and a divider or other signal conditioning is added where needed. We use GPIO 34 and GPIO 35 because they belong to ADC1, which can still be used while Wi-Fi is switched on. The voltage module we have is marked ET5204/C5204. Its exact pinout

Module pin	ESP32 / circuit	Note
ACS712 VCC	5 V rail	Powered from the LM2596
ACS712 GND	GND	Common ground
ACS712 OUT	GPIO 34	Rests near 2.5 V, check the range, condition if needed
ACS712 screw terminals	In series with the motor, before the switch	Carry the full motor current
Voltage sensor input	Across the 12 V supply	Voltage to be measured
Voltage sensor + / −	5 V rail / GND	Module supply
Voltage sensor S	GPIO 35	Check module and output range first

Table 3.4: Connections of the current and voltage sensors

and conversion ratio are still to be confirmed. We also do not use a ZMPT101B for this job, because that module is meant for AC mains voltage, not for the DC supply of the motor.

3.2.5 OLED and buzzer

The OLED is an I2C display, so it can share GPIO 21 and GPIO 22 with the MPU6050. The two devices have different addresses (0x68 for the MPU6050 and normally 0x3C for the common SSD1306 OLED), so they do not clash. The buzzer is driven from one free output pin of the ESP32, which will be fixed when the final board layout is made. The OLED and the buzzer are not yet included in Figure 3.2.

3.3 Control Part

The control part has two programs: the firmware on the ESP32 and the Python application on the computer. Figure 3.3 shows how the data move through them.

3.3.1 ESP32 firmware

The firmware is written in the Arduino IDE. At start-up it initialises the I2C bus, the one-wire bus, the ADC pins, the OLED and the Wi-Fi access point, and then it enters a loop. In each pass it reads the acceleration and gyroscope values from the MPU6050, the temperature from the DS18B20, and the raw ADC values of the current and voltage sensors. It checks that each sensor answered, updates the OLED and, if a simple limit is crossed (for example, a very high temperature), switches the buzzer on. The latest readings are held in memory and returned as JSON when a client asks for the data page of the small web server running on the board. The same server also serves the existing live-monitoring page.

3.3.2 Data format

The JSON reply has a fixed set of fields, so the Python side can check it easily. The fields used in the current firmware are listed in Table 3.5. Current and voltage are sent as raw ADC counts for now. The conversion to amperes and volts is done afterwards, once the calibration is finished, and the raw values are always kept in the saved data.

Field	Example	Meaning
`ax, ay, az`	0.15, 0.22, 1.03	Acceleration on three axes
`gx, gy, gz`	0.02, 0.01, 0.03	Angular rate on three axes
`temperature`	42.5	Motor housing temperature
`current_raw`	512	Raw ADC count of the ACS712
`voltage_raw`	2180	Raw ADC count of the voltage sensor
`mpu_ok, temperature_ok`	true	Sensor-valid flags
`uptime_s`	1250	Time since the board started

Table 3.5: Fields of the JSON record returned by the ESP32 (example values only)

3.3.3 Python application

The Python program works as the middle layer. It joins the ESP32 Wi-Fi network, asks for data at regular intervals, checks the HTTP status and the required fields, adds a time stamp and saves each good record as a row of a CSV file. Bad replies and lost connections are logged and skipped, so one failure does not stop the collection. For training, the program cuts the saved data into windows and computes features from each window. For live use, the same feature code is applied to the most recent window and the result is given to the trained models. It is important that training and live operation use *exactly* the same preprocessing, otherwise the model sees numbers it was never trained on. The main libraries are listed in Table 3.6.

Library	**Use**
requests	HTTP requests to the ESP32
pandas, numpy	Data handling, windows and features
scikit-learn	Random Forest training and prediction
joblib	Saving and loading the trained models
Flask or FastAPI	Optional API that returns predictions to the dashboard
matplotlib	Plots for analysis

Table 3.6: Python libraries used in the application

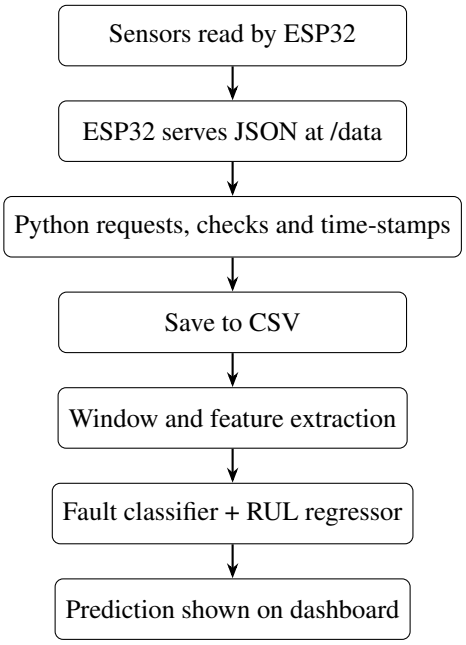


Figure 3.3: Flow of data through the control part

3.4 Power Circuit Part

There is no switching or power-electronics stage of our own in this project, so by "power part" we mean how the boards and the motor are supplied (Figure 3.4). The motor has its own DC supply. The logic side is fed from a supply through an LM2596 step-down converter, which brings the voltage down to 5 V for the VIN pin of the ESP32. The on-board regulator of the ESP32 makes the 3.3 V rail for the MPU6050, DS18B20 and OLED. The motor is never powered from an ESP32 pin.

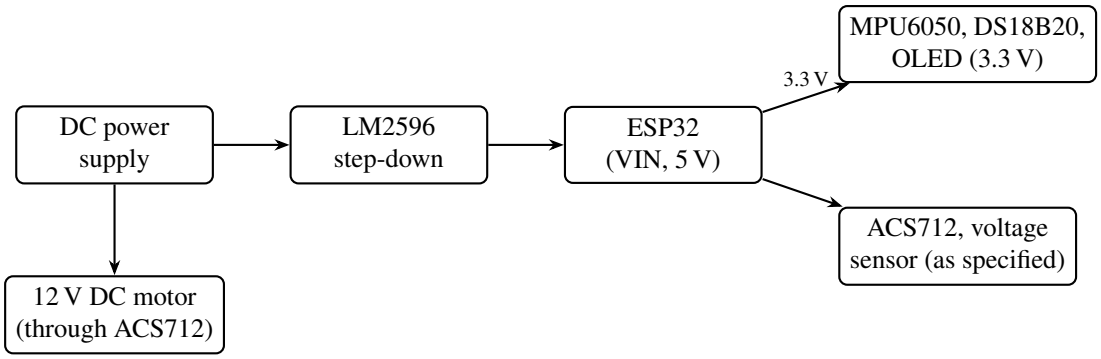


Figure 3.4: Power distribution in the monitoring unit

We keep the motor wiring and the logic wiring apart, and sensor wires are routed away from the high-current motor wires as far as possible, because a brushed motor produces a lot of electrical noise. All grounds are joined at a common point. Before the first power-up, the supply ratings, the wiring and the ADC voltage limits are checked once more.

3.4.1 Approximate power budget

Table 3.7 lists the approximate current of the logic blocks. The numbers are rounded typical values from the datasheets [14, 15, 16, 17] and not our own measurements. The real current of the finished unit will be measured later. In total the logic side stays well under 0.5 A, which is easy for an LM2596 module [18]. The motor is the real load, and its current goes through the ACS712, which is why the 20 A version has plenty of range for a 150 W, 12 V motor.

Block	Supply	Approximate current
ESP32 with Wi-Fi active	5 V (VIN)	about 80–240 mA, higher peaks
MPU6050	3.3 V	a few mA
DS18B20	3.3 V	about 1 mA
ACS712 board	5 V	about 10 mA
OLED	3.3 V	10–20 mA
Buzzer	5 V / pin	tens of mA while sounding

Table 3.7: Approximate current consumption of the logic blocks

3.5 Basic Parts and Their Working Principles

3.5.1 Choice of ESP32

The ESP32 is a cheap microcontroller with Wi-Fi, several ADC channels and hardware I2C, so one small board can read all the sensors and also host the web server. Its ADC is 12 bit, which gives counts from 0 to 4095 over a limited input range, and the inputs are not 5 V tolerant [14]. Part of its ADC cannot be used while Wi-Fi is on, which is the reason for choosing ADC1 pins. Keeping the heavy machine-learning work on the computer lets us change and retrain the models without touching the firmware.

3.5.2 MPU6050 and vibration sensing

The MPU6050 measures acceleration with a micro-machined (MEMS) accelerometer and gives the three axes through I2C [15]. Vibration shows up as a fast changing acceleration around a mean value. Our motor turns at about 2750 RPM, which is roughly 46 Hz, so the basic rotation frequency is low enough for this sensor, and an imbalance should show up near that frequency. Some faults leave their signature at much higher frequencies, and the limited bandwidth and sample rate of the MPU6050 may not capture them. We keep this limit in mind in the results and do not claim more than the sensor can see.

Different faults are expected to affect the vibration in different ways. Imbalance raises the vibration at the rotation frequency. Misalignment changes the pattern and brings harmonics. A damaged bearing changes the vibration energy and adds its own characteristic frequencies. A loose mounting makes the vibration larger and more irregular. These are general principles, and how clearly they appear in our own data has to be checked.

3.5.3 DS18B20 temperature sensor

The DS18B20 is a digital sensor that sends the temperature as a number over a one-wire line, with up to 12-bit resolution [16]. This avoids the calibration problems of analog sensors. It shows overheating and the slow warming of a loaded motor, but it measures the housing and not the winding, and temperature alone cannot identify every fault. It is therefore only one input among several.

3.5.4 ACS712 current sensor

The ACS712 passes the motor current through a copper path inside the chip and measures the magnetic field it makes with a Hall element. The output is a voltage that sits at about half the supply when the current is zero and moves up or down in proportion to the current. For the 20 A version the sensitivity is about 100 mV per ampere [17]. To convert a count to amperes we need the zero-current offset, the sensitivity, the ADC setting and some filtering of the noise from the brushed motor. In our earlier tests, the ADC stayed near a constant value even when the load changed. We have not yet found the cause, and it will be checked before the current is used as an input to any model: the supply to the module, the ground, the voltage at the OUT pin, the ADC pin, the series wiring and whether the code reports raw counts or calibrated current.

3.5.5 Voltage sensor

The voltage module works as a resistor divider that scales the 12 V supply down into the ADC range. The motor voltage is then

$$V_{motor} = V_{ADC} \times K \tag{3.1}$$

where K is the conversion ratio of the module. We will find K from the circuit values and confirm it with a multimeter, and not simply assume it.

3.5.6 Serial interfaces and communication

Two wired interfaces and one wireless link are used. The I2C bus connects the MPU6050 and the OLED to the ESP32 with just two wires, and each device has its own address on

the bus. The one-wire bus connects the DS18B20, with a pull-up resistor to hold the line high. The analog sensors use ADC pins directly. The link between the ESP32 and the computer is HTTP over Wi-Fi: the ESP32 acts as the server and Python sends a GET request and reads the JSON that comes back. HTTP is not the fastest option, but it is simple to test in a browser, and it is enough for readings that change slowly. Because each request returns only a single snapshot, fast vibration windows cannot be built from many polled requests. For that purpose the ESP32 will have to sample a short burst by itself and send the whole window at once, and we will decide this once the sample rate needed is clear.

3.5.7 Data preparation and feature extraction

A single acceleration reading says very little, so we work with short windows of samples. For each window we compute simple statistics for every axis: the mean, the standard deviation, the root-mean-square (RMS) value, the peak and the peak-to-peak value, together with the overall signal magnitude. When the sample rate is high enough, we also use the Fast Fourier Transform (FFT) to find how the energy is spread over frequency. Temperature, current and voltage, and how they change over time, are added to the same feature vector. The window length, the sample rate, the units and the handling of missing values are fixed once and used in the same way for training and for live use. In our earlier work on a public motor-fault dataset, adding FFT-based frequency features raised the accuracy of a Random Forest from 65 % to 94 %, which shows how much the features matter.

3.5.8 Machine-learning models

Random Forest classifier

A Random Forest builds many decision trees on random parts of the data and lets them vote, which makes it steady and needs little tuning [7]. We use it as the baseline for fault classification. The classes we aim for are normal operation, mechanical imbalance, horizontal and vertical shaft misalignment, and bearing ball, cage and outer-race faults. The model can only separate the classes for which it has enough correctly labelled examples, so the final list of classes depends on the data we can actually get. A model trained on a laboratory dataset such as MAFAULDA [10] will not work on our motor automatically,

because the sensors, the mounting and the operating conditions are different. It is used to build and test the pipeline first.

Random Forest regressor for RUL

Remaining Useful Life is the operating time left before a defined end-of-life condition is reached. A second Random Forest, this time a regressor, takes condition and degradation features over time and gives a number as output. For this we need data in which the remaining life is known, such as run-to-failure data. Fault labels alone do not provide that. Before training, we must therefore decide what counts as failure, which part's life is being estimated, in which unit the RUL is given and how the error will be judged. If the data come from bearings, the output is bearing RUL, and we do not call it the RUL of the whole motor unless there is evidence for it.

Performance metrics

For the classifier we use the confusion matrix and read from it the accuracy, precision, recall and F1-score of each class. For the RUL regressor we use the mean absolute error (MAE) and the root-mean-square error (RMSE), and we look at how the error changes with the operating condition. The data are split by recording session or run, and not by random rows, because neighbouring windows of the same run look almost the same, and a random split would make the results look better than they are. For a maintenance tool, a missed fault is worse than a false alarm, so we pay special attention to the recall of the fault classes and not only to overall accuracy.

3.5.9 Dashboard

The dashboard shows live temperature, vibration features, current and voltage with their units and a flag for each sensor, along with the connection status and the time of the last accepted sample. Once the models are trained and checked, it also shows the predicted condition and the RUL. A prediction can only be displayed if it reaches the page, so there are two options. In the first, the ESP32 keeps serving the existing page, and Python offers a small API with the latest result that the page asks for from time to time. In the second, Python serves the whole dashboard itself. Either way, the dashboard must show a clear

warning when the data are missing, old or outside the range the model was trained for, and must not show a made-up number.

3.5.10 Limitations and safety

We list the limits of the design openly. The models are proposed and are not yet trained, so no prediction is claimed in this chapter. Raw ADC counts are not amperes or volts until calibrated. Each fault class needs its own labelled data. The MPU6050 may miss high-frequency bearing signatures. RUL is an estimate and not a promised failure time. On the safety side, ratings, wiring and ADC limits are verified before power is applied, no hazardous voltage is connected to an ESP32 pin, and the tests are done in a controlled set-up in which no dangerous mechanical fault is created on purpose.

3.6 Summary

In this chapter we presented the block diagram and the proposed wiring, described the firmware and the Python application, and explained the power arrangement. We also described the working principles of the sensors, the feature extraction and the two Random Forest models, and the dashboard. The next chapter describes the hardware that implements this design and the experiments done so far.

Chapter 4

HARDWARE SETUP OF THE PROJECT

4.1 Overview

This chapter describes the hardware that puts the system design of the earlier chapters into practice. The set-up is built around a 12 V brushed DC motor, which is the machine being monitored, and an ESP32 board that reads four sensors: an MPU6050 for vibration, a DS18B20 for housing temperature, an ACS712 for motor current and a DC voltage sensor module for the supply voltage. The ESP32 serves these readings over its own Wi-Fi network, so a computer can collect them without a router or internet connection. We begin with the power circuit, where a single 12 V supply feeds the motor directly and the electronics through an LM2596 step-down converter, and the signal circuit, which connects each sensor to the ESP32 through I2C, one-wire or analogue inputs. Each component is then described in turn, covering its role, why it was chosen and its limits, followed by the step-by-step assembly, in which every stage was tested before the next was added so that faults could be traced easily. The chapter closes with the design points that decide whether the readings can be trusted on a running motor: the 3.3 V limit of the ESP32 analogue inputs, calibration of the current and voltage channels, the electrical noise of a brushed motor, rigid mounting of the vibration sensor and safe handling of the motor supply. Together, these show not only how the hardware was built but also how it must be set up and checked to produce reliable data for the machine-learning work in the next stage.

4.2 Introduction

The hardware has two sides. On the machine side there is a 12 V brushed DC motor (Yalu MY6812) that we want to keep an eye on. On the electronics side there are four sensors and one controller. An MPU6050 measures vibration, a DS18B20 measures temperature, an ACS712 measures the current drawn by the motor, and a DC voltage sensor module measures the supply voltage. All four are read by an ESP32, which creates its own Wi-Fi network and serves the readings as a small JSON message at the address /data. A Python program on a computer collects these readings, stores them, calculates features and runs the machine-learning models.The overall system architecture is shown in the block diagram in Chapter 3.

4.3 Hardware Setup

4.3.1 Power Circuit

The whole system runs from one 12 V DC source. Four things hang on it: the motor, which is connected through a switch and through the current-carrying terminals of the ACS712, the input terminals of the voltage sensor module, which simply look at the 12 V, the input of an LM2596 step-down converter, the common ground for everything. The LM2596 brings the 12 V down to 5 V. Its output feeds the 5 V (VIN) pin of the ESP32 and the supply pins of the sensor modules. The ESP32 board has its own regulator that makes the 3.3 V rail from this 5 V. The switch in the motor line lets us start and stop the motor without switching off the electronics, so the sensors can show the readings of the idle motor as well.

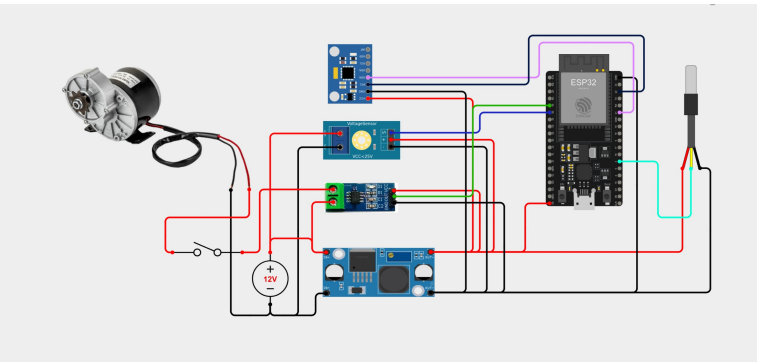


Figure 4.1: Wiring Diagram

4.3.2 Control Circuit

The signal connections are listed below. The MPU6050 talks to the ESP32 over I2C. The DS18B20 uses the one-wire protocol on a single data line. The ACS712 and the voltage sensor give analogue voltages that the ESP32 reads with its ADC.

ESP32 pin	Connected to	Function
GPIO 21	MPU6050 SDA (and OLED SDA)	I2C data
GPIO 22	MPU6050 SCL (and OLED SCL)	I2C clock
GPIO 4	DS18B20 DATA	One-wire data
GPIO 34	ACS712 OUT	Analog input (ADC1)
GPIO 35	Voltage sensor S	Analog input (ADC1)
5V (VIN)	LM2596 OUT+	Supply for the board
3V3	DS18B20 pull-up, OLED	Pull-up and power
GND	LM2596 OUT− and all sensor grounds	Common reference

Table 4.1: Signal connections between the sensors and the ESP32

4.4 Hardware Components

4.4.1 DC Motor

The motor under test is a Yalu MY6812 brushed DC motor with a small gearbox and sprocket. Table 4.2 gives the figures we have noted for it. We took them from earlier project notes, and they must be checked against the nameplate before the final report is submitted.

Parameter	Value
Model	Yalu MY6812
Type	Brushed DC motor
Nominal voltage	12 V
Rated power	about 150 W [confirm on nameplate]
Speed	about 2750 RPM [confirm on nameplate]

Table 4.2: Main specifications of the motor

4.4.2 ESP32 Development Board

The ESP32 is the link between the sensors and the computer. It reads the sensors, formats the data, creates the Wi-Fi access point and answers HTTP requests. In the first stage it

does not run the machine-learning models. Those run in Python on the computer, which keeps the firmware simple and lets us change the models without reprogramming the board. Two points about the board need to be settled in the final report. Our wiring diagram and pin list are for the ESP32 DevKit (WROOM-32) and use GPIO 34 and 35, which are input only ADC pins. The block diagram, however, was labelled ESP32-C3. These are different chips with different pin numbers, so the label of the block diagram should be corrected to match the board actually used [confirm the board]. Another useful point is that GPIO 34 and 35 belong to ADC1, which keeps working while Wi-Fi is on. The other ADC group, ADC2, does not.

4.4.3 MPU6050 Vibration Sensor

The MPU6050 contains a three-axis accelerometer and a three-axis gyroscope. We use mainly the acceleration, because vibration shows up there first. With the AD0 pin tied to ground the sensor answers at I2C address 0x68, and our scan detected it at exactly that address. Its limits matter for the project. It samples at a modest rate and its bandwidth is limited, so it can follow imbalance and misalignment type vibration reasonably well, but fine bearing signatures at high frequency may be out of its reach. We will say this openly when we report results.

4.4.4 DS18B20 Temperature Sensor

The DS18B20 is a digital temperature sensor that works on a single data line. We use the waterproof probe version and fix it against the motor body with thermal paste and a clamp. It tells us about heating of the housing, for example a slow rise during a long run or a faster rise under load. It does not measure the winding temperature inside the motor, and we do not claim that it does.

4.4.5 ACS712 Current Sensor (20 A)

The ACS712 is a Hall-effect current sensor. The motor current flows through its two heavy screw terminals, in series with the motor supply, and the sensor produces a small voltage on its OUT pin that follows the current. At zero current the output sits at half the supply voltage, and for the 20 A version it changes by about 100 mV per ampere. The

motor current itself never goes to the ESP32. Only this low-voltage signal does, which is why the sensor also gives electrical isolation of the measurement from the power path.

4.4.6 DC Voltage Sensor Module

This module is a resistor divider on a small board, marked for inputs below 25 V. The 12 V supply goes to its screw terminals and a scaled-down voltage comes out of the S pin. On the common 0 to 25 V boards the divider ratio is 5:1, which would turn 12 V into about 2.4 V and keep it inside the ADC range. Our module is referred to as ET5204/C5204, and since we have not yet confirmed its ratio from the datasheet or by measurement, we treat the factor K in V_motor = V_adc × K as unknown until we do [measure K].

4.4.7 LM2596 Step-Down Converter

The LM2596 is a switching regulator that accepts a higher input voltage and gives an adjustable lower output, up to 3 A. We use it only for the electronics, which draw a small fraction of that, so it runs cool and with a lot of margin. Because it is a switching converter, it adds some high-frequency noise to the 5 V rail, and we keep this in mind when reading the analogue sensors.

4.4.8 Other Components

The remaining parts are a toggle switch in the motor line, a 4.7 k resistor for the DS18B20 data line, jumper wires, a breadboard or perforated board, a USB cable for programming the ESP32, a 12 V supply of adequate current rating, and the planned OLED display and buzzer. The sensor specifications are collected in Table 4.2.

4.5 Complete Hardware Setup

We built the set-up in stages so that every stage was proved before the next was added. When something fails, this makes it clear which part is at fault. 1. Power supply: The LM2596 was set to 5 V with a multimeter and nothing connected to it. Only then was the ESP32 connected.

Part	Key figures	Supply
MPU6050	3-axis accelerometer (± 2 to ± 16 g), 3-axis gyroscope, I2C	3.3 V (module may accept 5 V)
DS18B20	-55 to $+125$ °C, about ± 0.5 °C in the mid range, 9 to 12-bit	3.0 to 5.5 V
ACS712-20A	± 20 A, about 100 mV/A, zero-current output at half supply	5 V
Voltage module	0 to 25 V input, divider output [ratio to be verified]	none needed on the divider
LM2596 module	input up to about 40 V, adjustable output, 3 A maximum	12 V input

Table 4.3: Main figures of the sensors and the converter (from manufacturer datasheets)

2. ESP32 alone: The board was programmed from the Arduino IDE and its Wi-Fi access point, named Motor_Monitor, was checked from a laptop.

3. MPU6050: It was wired to GPIO 21 and 22 and found at address 0x68 with an I2C scan. Acceleration values then appeared on the serial monitor.

4. DS18B20: It was connected to GPIO 4 and gave sensible room temperature readings.

5. Web page and /data: The ESP32 served the live dashboard and the JSON data. The keys we used so far are mpu_ok, temperature_ok, ax, ay, az, gx, gy, gz, temperature, current_raw, voltage_raw and uptime_s.

6. ACS712 and voltage sensor: Both were wired to their ADC pins. The current sensor reading stayed almost constant when the load changed, and this is under investigation (Section 5.3). The voltage sensor still needs its ratio confirmed.

7. Motor in the loop: The motor is connected through the switch and the ACS712 only after steps 1 to 6 are reliable, and the first runs are done with the motor unloaded.

4.6 Hardware Design Considerations

4.6.1 Pin Allocation

Table 4.3 summarises the ESP32 pins in use. All other pins are free for later additions such as the OLED and the buzzer. The OLED can share the I2C pins with the MPU6050, because the two have different addresses (0x3C or 0x3D for the display, 0x68 for the MPU6050). The buzzer needs one free output pin [assign, for example GPIO 25] and is

best driven through a small transistor.

ESP32 pin	Used for	Type
GPIO 21	MPU6050 SDA	I2C
GPIO 22	MPU6050 SCL	I2C
GPIO 4	DS18B20 data	One-wire
GPIO 34	ACS712 OUT	ADC1, input only
GPIO 35	Voltage sensor S	ADC1, input only
5 V / VIN	From LM2596	Power
GND	Common ground	Power

Table 4.4: Summary of the pin allocation

4.6.2 ADC Limits and Signal Conditioning

This is the most important design point in the whole circuit. The ESP32 pins are not 5 V tolerant, and the usable ADC range is only about 0 to 3.3 V. The ACS712 is powered from 5 V, so at zero current its output is already near 2.5 V and it climbs by 100 mV for each ampere. It reaches 3.3 V at about 8 A, while a 150 W motor at 12 V draws around 12 A when loaded, and more when it starts. In our wiring diagram the OUT pin goes straight to GPIO 34, so before running the loaded motor we plan to add a resistor divider, for example $10\,\mathrm{k}\Omega$ and $18\,\mathrm{k}\Omega$, which scales 5 V down to about 3.2 V [confirm values and re-calibrate]. The ADC also needs calibration, because the ESP32 converter is not perfectly linear and the sensor has a zero-current offset. The same care applies to the other lines. In the diagram the MPU6050 and the DS18B20 take their power from the 5 V rail. Their data lines must still idle at 3.3 V, so the DS18B20 pull-up resistor goes to the 3V3 pin and not to 5 V, and the I2C lines should be checked with a multimeter. Powering both from 3V3, as in our original pin plan, removes any doubt. The wiring diagram does not show the $4.7\,\mathrm{k}\Omega$ pull-up, and it must be added.

4.6.3 Grounding, Noise and the Motor

A brushed motor is a noisy neighbour. Its brushes produce sparks and voltage spikes that travel along the supply and radiate into nearby wires. To keep the readings clean we plan to:

1) keep sensor wires short and away from the motor and the thick current wires, join all grounds at one point, with the motor current return not passing through the sensor

ground wires,

2) add a 100 nF ceramic capacitor across the motor terminals, and a diode across the motor if the supply allows it,

3) add a small capacitor at the sensor outputs near the ESP32 and average several ADC samples in software.

4.6.4 Sensor Mounting

Vibration data are only as good as the mounting of the sensor. The MPU6050 is fixed rigidly to the motor housing or the motor base with screws or strong adhesive, with its axes marked, and it is not left hanging on wires. The same mounting position is used for every recording, because moving it changes the signal and would make the training data inconsistent. The DS18B20 is clamped to the motor body in a fixed spot.

4.6.5 Safety and Handling

The motor supply is only 12 V, but a 150 W motor can draw a large current, so wires and connectors must be rated for it and the supply should be fused. The motor is never driven from an ESP32 pin. The electronics are checked once more before power is applied, and the motor is first run unloaded. We do not create dangerous mechanical faults on purpose. Any imbalance or misalignment will be small, controlled and done with the motor guarded and switched off while it is set up.

4.7 Summary

This chapter described the power circuit, the signal connections, the components, the staged assembly and the main design considerations of the hardware. The ESP32 reads vibration, temperature, current and voltage and serves them over Wi-Fi. Two things still need care before the data can be used for training: the ADC range of the current and voltage channels, and the calibration of both.

Chapter 5

Project Status Completed in S7

This chapter summarises what was done in the seventh semester. In short, the monitoring side of the project is built and partly verified, and the machine-learning side is planned for the eighth semester. We have tried to say clearly which items are finished and which are still being checked.

5.1 Activities Completed

The activities are listed in Table 5.1.

Sl.	Activity	Outcome	Status
1	Topic selection and problem identification	Need for condition-based maintenance of a DC motor defined; objectives fixed	Completed
2	Literature survey	Related work on motor faults, vibration analysis and RUL reviewed	Completed
3	System design and block diagram	Motor, four sensors, ESP32, Python, ML and dashboard architecture fixed	Completed
4	Component selection	ESP32, MPU6050, DS18B20, ACS712, voltage sensor and LM2596 chosen	Completed
5	MPU6050 and DS18B20	MPU6050 detected at 0x68; DS18B20 tested	Completed
6	ESP32 access point and dashboard	Motor_Monitor network and live web page with `/data` endpoint working	Completed

Table 5.1: Project status completed in the seventh semester

5.1.1 Observations

The completed work showed that the monitoring side of the project is on firm ground. Fixing the objectives and reviewing the literature first gave a clear reason for each design choice, and this carried into the system design and component selection. The ESP32, MPU6050, DS18B20, ACS712, voltage sensor and LM2596 were chosen as low-cost modules that can be wired quickly and replaced easily, which suits a test bench where the wiring changes often.

Building the set-up one stage at a time paid off. Because each sensor was added only after the previous one worked, any fault could be traced to a single part. The MPU6050 was detected at I2C address 0x68 in a bus scan, and acceleration values then appeared on the serial monitor. The DS18B20 gave sensible room-temperature readings on its one-wire line. The ESP32 also created its own Motor Monitor Wi-Fi network and served a live web page and the data endpoint, so readings can be collected at the test bench without a router or internet connection.

We also became clear about what these sensors can and cannot tell us. The MPU6050 has limited bandwidth, so it suits imbalance and misalignment type vibration better than fine, high-frequency bearing signatures. The DS18B20 measures the temperature of the motor housing, not of the windings inside. We will state both limits openly when the results are reported.

5.1.2 Summary

Six of the planned activities were completed in the seventh semester: topic selection, literature survey, system design, component selection, testing of the MPU6050 and DS18B20, and the ESP32 access point with its live dashboard. Together they give a working monitoring base, with vibration and temperature readings available wirelessly through the data endpoint. The remaining work, namely the current and voltage channels, data logging, fault dataset, classifier and RUL model, is planned for the eighth semester.

Chapter 6

Project status to be completed in S8

This chapter sets out the work planned for the eighth semester. The monitoring hardware built in the seventh semester provides the base: the sensors are wired to the ESP32, the MPU6050 and DS18B20 have been tested, and the ESP32 serves live readings over its own Wi-Fi network through the data endpoint. What remains is mainly on the software side. This includes the Python receiver with CSV logging, the fault dataset, the fault classifier, the RUL estimation model and live prediction. The verification and calibration of the current and voltage channels also has to be finished, since the training data depends on them. The remaining activities are listed in Table 6.1, followed by the approach for each.

6.1 Planned activities

Table 6.1 lists the work planned for the eighth semester. The sensing hardware is already built, so the plan starts by making the current and voltage channels trustworthy and then moves to data collection and machine learning.

6.2 Detailed work plan

6.2.1 Completion of the hardware

- **Current channel:** The near-constant ACS712 reading will be traced step by step: supply and ground of the module, the voltage on the OUT pin measured with a multimeter, the motor current passing through the IP+ and IP− terminals, and the

Sl.	Period	Planned activity
1	October 2026	Hardware: finding the cause of the constant ACS712 reading, adding the resistor divider, calibration of the current and voltage channels; Software: Python receiver and CSV logging
2	November 2026	Recording of healthy-motor data at different loads; controlled fault recordings; feature extraction and dataset preparation
3	December 2026	Training and comparison of the fault classifier; development of the RUL estimation model; first performance evaluation
4	January 2027	Live prediction on the dashboard, OLED and buzzer indication; testing, improvements and incorporation of feedback; research paper and final report
5	March 2027	Final demonstration, viva and project submission

Table 6.1: Project status to be completed in the eighth semester

scaling of the 12-bit ADC counts. A resistor divider will then be added so that the output stays below 3.3 V at the full motor current, and the channel will be calibrated against a reference ammeter.

- **Voltage channel:** The divider ratio K of the voltage module will be measured with a multimeter, and the ADC readings will be calibrated against it.
- **Noise reduction:** The capacitor across the motor terminals, single-point grounding, short sensor wires and averaging of several ADC samples will be applied, and the effect will be checked on the idle and running motor.
- **Local indication:** The OLED display and the buzzer will be wired to the ESP32 and tested, so that a warning is visible at the motor without a computer.

6.2.2 Software development

- **Python receiver and logging:** A Python program will read the data endpoint of the ESP32 at a fixed rate and store each reading with a timestamp in a CSV file, together with the operating condition and the fault label.
- **Data collection:** Recordings will be made for the healthy motor at several loads and for a small number of controlled faults, such as a small added imbalance or a

slight misalignment. The sensor mounting will be kept identical for every recording.

- **Features and fault classifier:** Time-domain features such as RMS, peak, kurtosis and standard deviation of the acceleration, together with the mean current, voltage and temperature rise, will be computed over fixed windows. Classifiers such as a random forest or a support vector machine will be trained and compared using a separate test set.

- **RUL estimation:** A health indicator will be built from the features and followed over time, and a trend-based or regression model will estimate the remaining useful life from it. Since run-to-failure data of a real motor cannot be recorded in the available time, the limits of this approach will be stated clearly.

- **Live prediction and dashboard:** The trained models will be called from the Python program on the live data, and the predicted condition and RUL will be shown on the dashboard and sent back to the ESP32 for the OLED and the buzzer.

6.2.3 System integration

The manual checks used so far will be replaced by an automatic chain: sensors, ESP32, Wi-Fi, Python receiver, models, dashboard and local alarm. The data format of the data message will be kept fixed so that the hardware and the software can be developed separately. The behaviour when a reading is missing or the Wi-Fi link drops will be defined, for example by skipping the window and logging the gap.

6.2.4 Performance evaluation

The following evaluations will be carried out.

- **Classifier:** confusion matrix, per-class precision, recall and F1-score and overall accuracy on a held-out test set.

- **Sensor channels:** error of the calibrated current and voltage readings against reference instruments, and the noise level with the motor running.

- **RUL model:** error of the estimated RUL against the known degradation progress of the test runs.
- **Latency:** time from a sensor reading to the update of the dashboard and the alarm.
- **Robustness:** behaviour of the system when the load, supply voltage or sensor position changes.

6.2.5 Improvement and optional extensions

Depending on the results and the time available, the following improvements are considered.

- Running a light-weight model directly on the ESP32, so that fault detection works without a computer.
- Using the motor current signature as an additional fault indicator.
- Adding a second motor or a different load to check how well the models carry over.
- Logging the data to a database for long-term trend analysis.

6.2.6 Documentation

A research paper will be prepared, and the final report will include the complete results, the circuit diagrams, photographs of the prototype and the source code of the firmware and the Python programs.

6.3 Risks and mitigation

Table 6.2 lists the main risks that can affect the completion of the work, with the planned actions.

Risk	Effect	Mitigation
Current channel cannot be made reliable	Missing load information in the features	Check the module against a reference ammeter; replace the sensor if it is faulty
ADC overload on the current or voltage input	Damage to the ESP32 or clipped readings	Resistor divider, measured with a multimeter before connecting the motor
Motor noise on the analogue and I2C lines	Unstable readings, poor features	Short wires, single-point ground, capacitors, averaging in software
Limited bandwidth of the MPU6050	Fine bearing faults not detected	Restrict the claims to imbalance and misalignment type faults
Few fault recordings or unbalanced classes	Low accuracy, poor generalisation	Repeat recordings, use cross-validation, report per-class metrics honestly
No real run-to-failure data	RUL model cannot be validated fully	Use a health-indicator trend, state the limits of the estimate
Sensor mounting changes between recordings	Inconsistent training data	Fixed mounting position, marked axes, rigid fixing
Software work taking longer than planned	Delay in testing and the report	Start the receiver and logging early; keep the `/data` format fixed
Mechanical or electrical hazard	Injury or damage	Fused supply, guarded motor, only small controlled faults, motor first run unloaded

Table 6.2: Risks and their mitigation

6.4 Future scope

The system can be extended with a sensor of higher bandwidth, so that bearing faults can also be detected, and with models that run on the ESP32 itself. It can be applied to other motors and machines, and combined with a cloud dashboard for monitoring several machines from one place. With data from real run-to-failure tests, the RUL model

could be validated and become a useful tool for condition-based maintenance in small workshops and industries.

6.5 Conclusion

An AI-based predictive maintenance system for a DC motor has been proposed. The problem, the literature, the methodology and the hardware design have been completed, the vibration and temperature channels have been tested, and the ESP32 serves live readings over its own Wi-Fi network. The calibration of the current and voltage channels, the data logging, the fault classifier, the RUL model and the live prediction are planned for the eighth semester, so that the real performance of the system is known.

Bibliography

[1] S. Nandi, H. A. Toliyat and X. Li, "Condition monitoring and fault diagnosis of electrical motors—a review," *IEEE Trans. Energy Convers.*, vol. 20, no. 4, pp. 719–729, 2005.

[2] A. K. S. Jardine, D. Lin and D. Banjevic, "A review on machinery diagnostics and prognostics implementing condition-based maintenance," *Mech. Syst. Signal Process.*, vol. 20, no. 7, pp. 1483–1510, 2006.

[3] Y. Lei *et al.*, "Machinery health prognostics: A systematic review from data acquisition to RUL prediction," *Mech. Syst. Signal Process.*, vol. 104, pp. 799–834, 2018.

[4] W. Zhang, D. Yang and H. Wang, "Data-driven methods for predictive maintenance of industrial equipment: A survey," *IEEE Syst. J.*, vol. 13, no. 3, pp. 2213–2227, 2019.

[5] S. Zheng, K. Ristovski, A. Farahat and C. Gupta, "Long short-term memory network for remaining useful life estimation," in *Proc. IEEE Int. Conf. Prognostics and Health Management*, 2017.

[6] S. M. A. Cruz and A. J. M. Cardoso, "Stator winding fault diagnosis in three-phase synchronous and asynchronous motors by the extended Park's vector approach," *IEEE Trans. Ind. Appl.*, vol. 37, no. 5, pp. 1227–1233, 2001.

[7] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[9] Y. Lei *et al.*, "Applications of machine learning to machine fault diagnosis: A review and roadmap," *Mech. Syst. Signal Process.*, vol. 138, 106587, 2020.

[10] Federal University of Rio de Janeiro, Signals, Multimedia and Telecommunications Laboratory, "MAFAULDA: Machinery Fault Database."

[11] P. Nectoux *et al.*, "PRONOSTIA: An experimental platform for bearings accelerated degradation tests," in *Proc. IEEE Int. Conf. Prognostics and Health Management*, 2012.

[12] A. Saxena, K. Goebel, D. Simon and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *Proc. Int. Conf. Prognostics and Health Management*, 2008.

[13] Analog Devices, "ADXL345 3-axis digital accelerometer," datasheet.

[14] Espressif Systems, "ESP32 series datasheet."

[15] InvenSense, "MPU-6000 and MPU-6050 product specification."

[16] Maxim Integrated, "DS18B20 programmable resolution 1-Wire digital thermometer," datasheet.

[17] Allegro MicroSystems, "ACS712 fully integrated, Hall effect-based linear current sensor IC," datasheet.

[18] Texas Instruments, "LM2596 SIMPLE SWITCHER 3-A step-down voltage regulator," datasheet.



Department of Electronics and Communication Engineering
Vidya Academy of Science & Technology
Thalakkottukara, Thrissur - 680 501
(http://www.vidyaacademy.ac.in)